Lecture – Paging implementation

"I'm writing a book. I've got the page numbers done."

- Steven Wright

Virtual memory, caching, and page tables

- Read chapter 5.8
- Learning objectives
  - Upper level(s) of memory hierarchy use virtual addresses
  - Virtual memory addresses translated to physical between two levels and no later than just before main memory
  - Physical memory conceptually divided into sections for O/S, page tables, page frame storage

Processes use virtual addresses

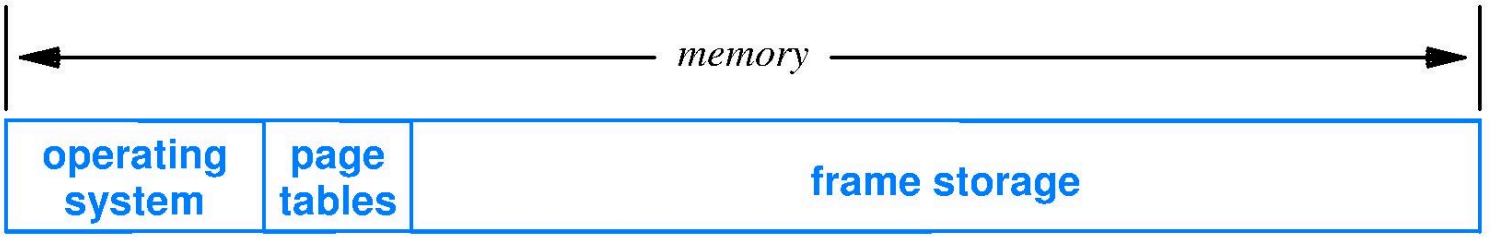
- Every process uses the same set of virtual addresses, e.g., 0x0000000000000000 to 0xFFFFFFFFFFFFFFFF (64-bit address space)
- How to distinguish a block of one process from a block of another?

Disambiguating virtually addressed blocks

- Two approaches
  - O/S invalidates all such blocks when switching execution from one process to another
  - Cache hardware augmented to include process ID number alongside the tag

Where are page tables stored?

- Conceptually, partition physical memory into three parts
  - Largest area is for page frames



How to build a demand paging system?

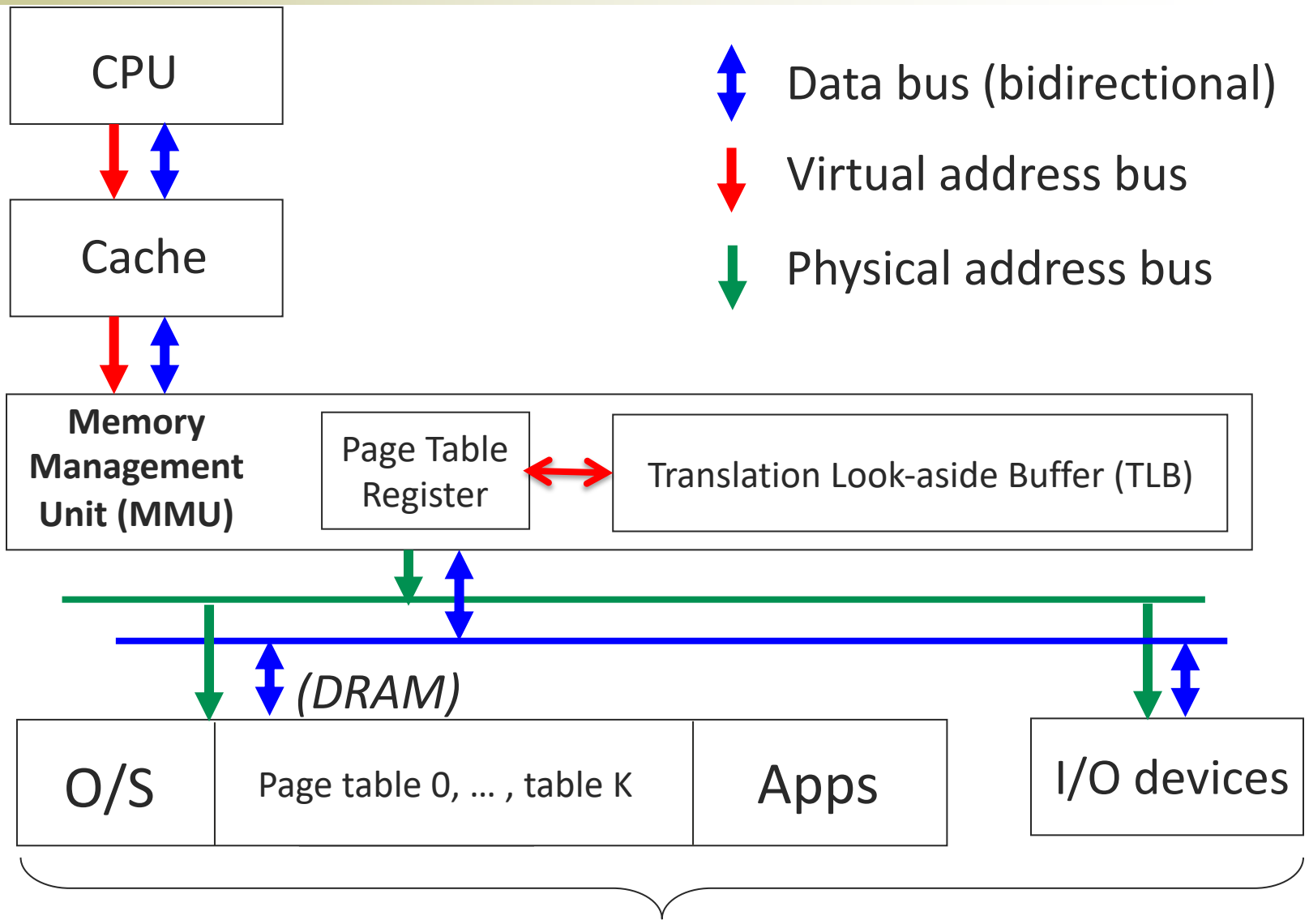
- Read chapter 5.8
- Learning objectives
  - Explain how virtual memory increases the time cost of all physical memory accesses
  - Explain the MMU
  - Explain the construction and operation of the translation lookaside buffer

Performance problem with paging

- Paging adds indirection to memory access
  1. Lookup translation of virtual address to physical address: access in DRAM the page table of this process
     - Where is this page table? Look up in table of tables?!
  2. Using translation, access DRAM physical memory

- The above makes computers too slow
- Use a Memory Management Unit (MMU)
  - HW to speedup the translation process

MMU in the memory hierarchy



MMU components

- Page Table Register
  - Stores physical address pointer to page table for current process
  - Each process has its own page table in physical memory
  - Page table register updated by O/S when new process launches
- Cache of recent virtual to physical translations
  - Saving recent translations, (virtual page #, physical page frame), in a small, fast cache memory

Design of a translation cache

- Compulsory miss rate will be low, inherently
  - One page contains many words and program has locality of reference at the word level
- Capacity needed is small due to locality of reference
  - “Working set” of pages is a smallish integer
- Conflict miss rate can be high if translation cache is direct mapped
  - Active pages come from several widely spaced locations in virtual address space – text, data, stack, libraries
- Fully associative translation cache design will transform conflict misses into capacity misses
  - Capacity does not need to be large anyway

CAM – a memory for fully associative

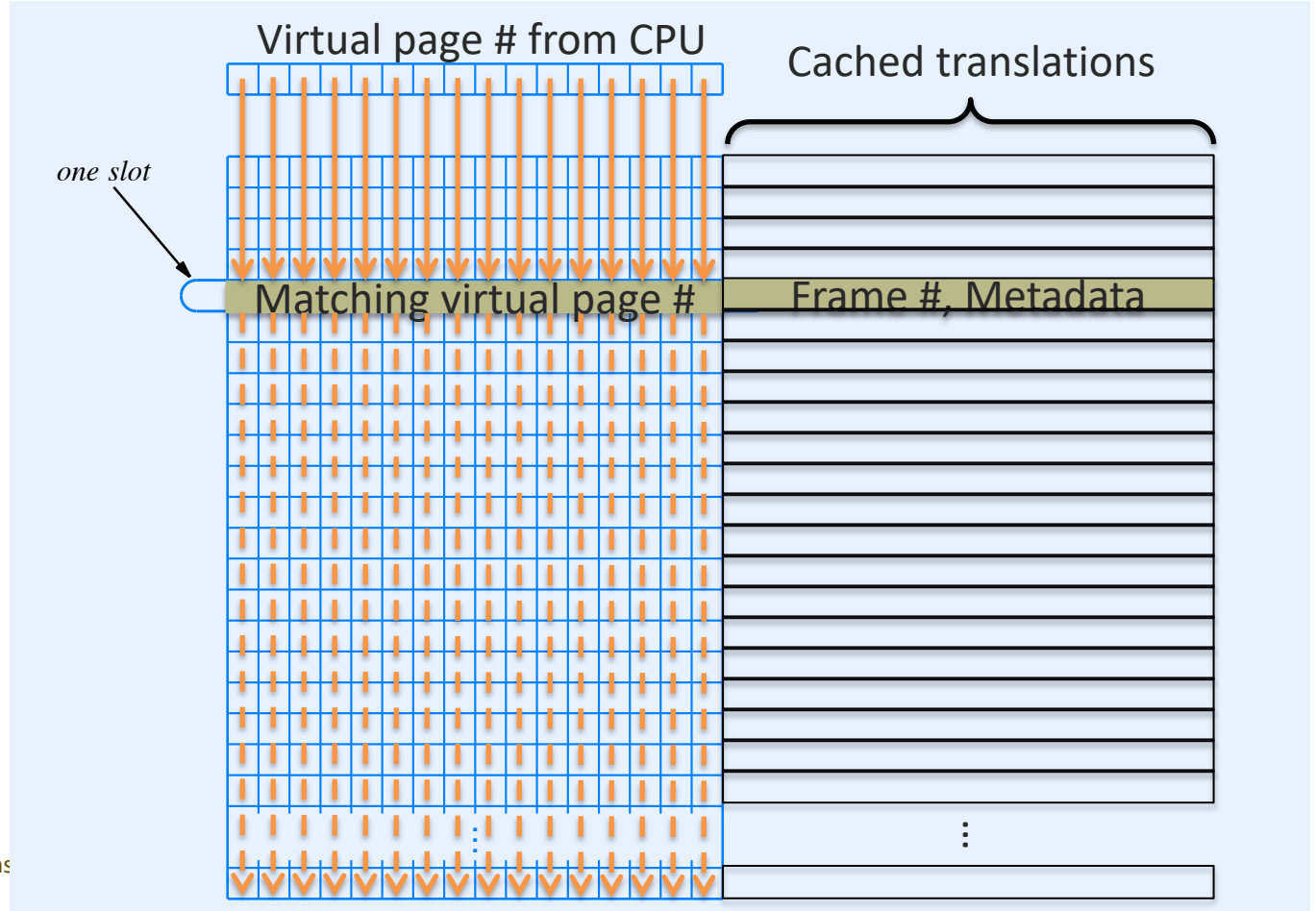
- Content addressable memory is also called *associative memory* (see text chapter 11.26)
- CAM memory cell includes latch to store one bit and a dedicated bit comparison circuit
- CAM organized as N m-bit slots, where N and m chosen by need and cost

Searching CAM takes constant time

- CAM does not use pointers, instead
  - Present key (k-bit string) to CAM
  - All slot circuits simultaneously test their content for key match, searching the entire CAM
  - Result is delivered by a wire from each slot, asserted where key is found, if found

Translation Lookaside Buffer

- CAM search hit returns frame # and metadata
- If CAM miss; use page table in DRAM



TLB and CAM

- Nearly 100% TLB hit rate when CAM capacity is somewhat in excess of the number of pages in the program working set

- Use CAM when search performance is everything, thus justifying high circuit cost

How to reduce page table size?

- Learning objectives
  - Explain how a multi-level page table works
  - Explain how a multi-level page table saves space

Page table can be very large

- Example virtual memory system
  - 48-bit virtual addresses (VA)
  - 4 Kbyte page size (12-bit offset field size)
  - 36-bit physical address (up to 2^{36} bytes = 64 GB DRAM)
  - 24-bit page frame # (∵ 36 bits – 12 bits = 24-bits)
- How many virtual pages?
  - 2^{36} virtual pages
    - From 2^{48} VAs / 2^{12} VAs per virtual page = 2^{36} virtual pages
- What is the size of the page table?
  - If 4-byte table entry from 24-bit frame # + 8 bits metadata
  - Then page table is 2^{36} entries x 2^{2} bytes/entry = 256 GiB !

Page table usually is very, very, very empty

- Virtual address space is enormous, most programs use only a tiny, tiny, tiny portion of that space
  - Thus, most table entries say only “Not resident”
  - Table is an array data structure
  - Array with many repeated entries is space inefficient
- Seek a space-efficient data structure, guided by
  - Retain fast access using virtual address page# bits
  - Use the facts of program use of virtual address space
    - Gaps between program segments represent long sequences of virtual pages not used, thus all are never resident
    - There are large gaps between heap, libraries, stack segments

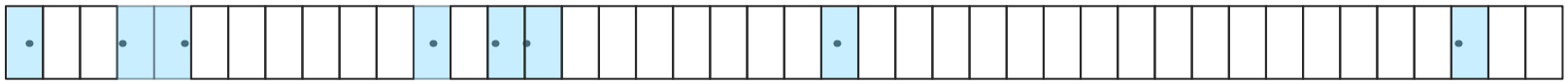
Most virtual pages are not resident

- Page table array diagram showing Resident == True with .

- The page table array contains very little information relative the the bytes of memory occupied
- Compress the array by removing large stretches of consecutive Resident == False entries
- 

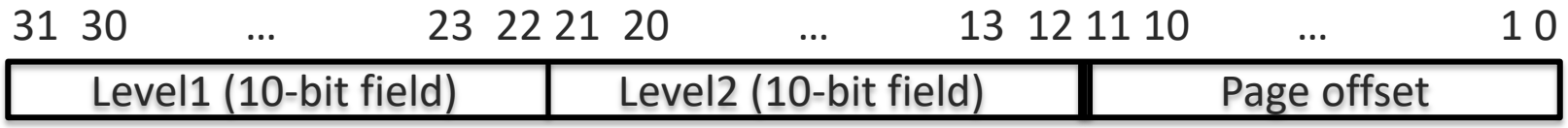
Compress by replacing array with a tree

- 1-dimensional page table is a sparse array

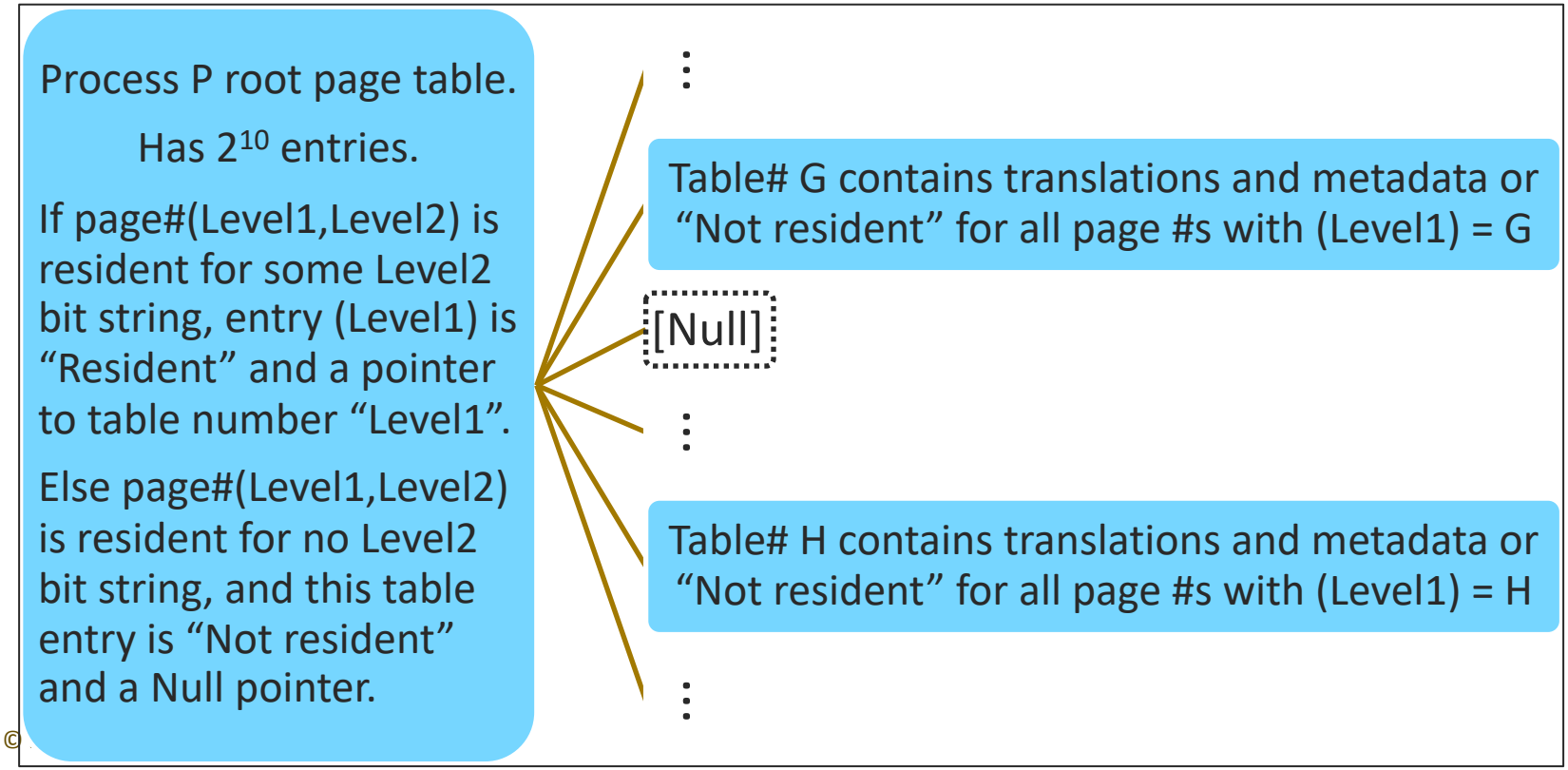
- Consider a k-level tree with all internal nodes of degree 2^((# bits in VPN)/k)
- Successive leaf nodes assigned successive 2^((# bits in VPN)/k) entries of page table array
- Leaf node (box in diagram below) is empty (no data) if all its pages are non-resident; holds 2^((# bits in VPN)/k) entries if >0 pages resident (blue shading)
- 

Replace sparse array with tree of small, dense arrays

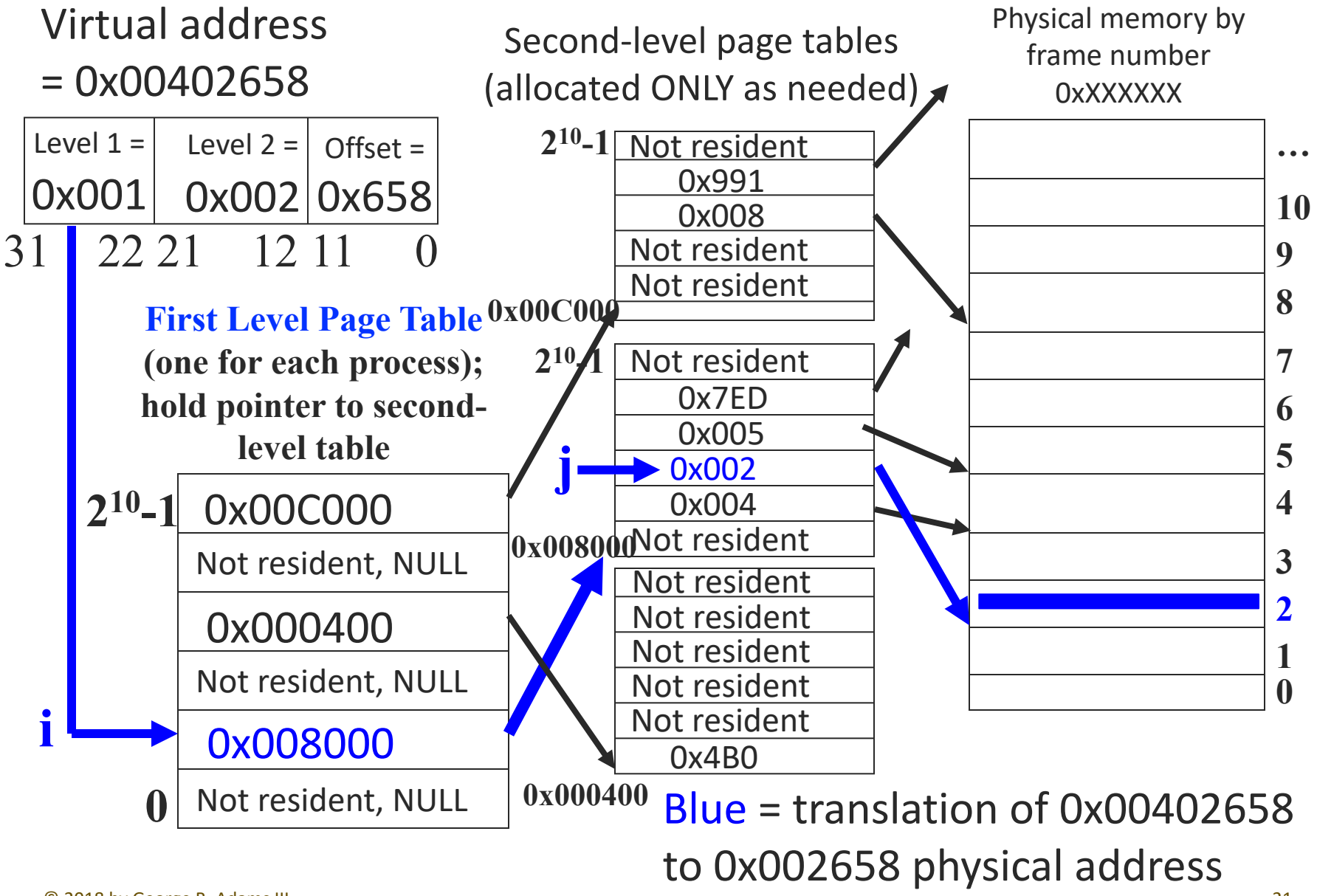
- Split page# field into k fields, example here, 2 fields



- Page table becomes tree; only shaded portions use memory



2-level page table example translation



How is a page fault handled?

- Learning objectives
  - Explain how a page fault is handled

Types of page fault

- CPU tries to access a page not resident
  - In page table, Resident bit marked “No”
  - MMU signals O/S kernel to load page from disk

- CPU tries a memory access that violates page access metadata
  - MMU signals O/S, which stops the process with a SEGV or other error message

Handling a page fault for non-resident

1. CPU read or write at address in a non-resident page
2. MMU
    1. Performs translation
    2. Finds resident bit is not set
    3. Sends page fault signal (miss) to operating system (O/S)
3. O/S
    1. Changes CPU mode to O/S privilege level
    2. Saves information needed to restart process after fault: current_instruction_pointer (PC), all CPU registers
    3. O/S calls Page Fault Handler

Handling a non-resident page fault (page 2)

5. Page fault handler
    1. Uses replacement policy to identify a page frame in physical memory to hold incoming page
        1. If no free page frames, then select one to re-use
        2. If selected frame contains a dirty (changed) page, then write that page to disk now
    2. Load page from disk into chosen page frame
    3. Return chosen page frame to O/S
6. O/S
    1. Updates the translation table entry
        1. Page frame number
        2. Metadata bits (e.g., clear the modified bit, etc.)
    2. Restores process restart information
    3. Changes CPU mode and returns to process

Handling a non-resident page fault (page 3)

7. CPU restarts Fetch-Execute of faulting instruction
8. MMU caches translation in TLB

- The page fault is completely transparent to the program, that is, the program will have no knowledge that the page fault occurred
- Page fault elapsed time
  - ~10 million nanoseconds using a hard disk
  - ~100,000 nanoseconds using a fast SSD

Handling a page fault for access violation

1. CPU attempts read or write that is inappropriate
2. MMU
    1. Performs translation
    2. Finds access would violate metadata
    3. Sends access violation signal to O/S
3. O/S
    1. Kills process
    2. Sends violation signal

Summary

- Demand paged virtual memory
  - Is a form of caching
  - Lets main memory be shared in many ways
  - Speeds creation of child processes, allocation of zero-initialized memory
- K-level page tables strongly conserve space
- Handling a page fault takes a lot of time and energy

Example – Reducing power with SW

How reduce power with SW?

- Learning objectives
  - List several ways in which SW makes decisions that reduce energy use
  - Explain processor idle mode and when to use it

Macbook Air 11” battery tech spec

Battery and Power[2]

7 hours battery claimed in mid-2012 when I bought one.

2014 and Macbook Air 11” HW unchanged, yet 9-hour battery claimed.

Test conditions identical.

How can 9 hours be true? Answer: new operating system version:
OS X Yosemite

Up to 9 hours wireless web

Up to 9 hours iTunes movie playback

Up to 30 days standby time

Built-in 38-watt-hour lithium-polymer battery

45W MagSafe 2 Power Adapter with cable management; MagSafe 2 power port

2. Testing conducted by Apple in March 2014 using preproduction 1.4GHz dual-core Intel Core i5-based 13-inch MacBook Air units and preproduction 1.4GHz dual-core Intel Core i5-based 11-inch MacBook Air units. The wireless web test measures battery life by wirelessly browsing 25 popular websites with display brightness set to 12 clicks from bottom or 75%. The HD movie playback test measures battery life by playing back HD 720p content with display brightness set to 12 clicks from bottom or 75%. The standby test measures battery life by allowing a system, connected to a wireless network and signed into an iCloud account, to enter standby mode with Safari and Mail applications launched and all system settings left at default. Battery life varies by use and configuration. See www.apple.com/batteries for more information.

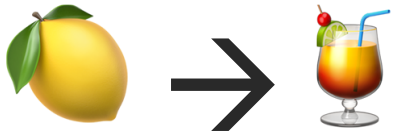
Energy saving examples in OS X Yosemite

1. Reduce page fault handling energy use
2. Reduce GUI energy use
3. Increase CPU idle time

From *OS X Yosemite Core Technologies Overview*, Apple, Inc., Oct. 2014, pp. 7-9.

Page fault handling energy and time use

- If all frames occupied, must replace page in memory
  - Page to replace may be dirty
    - **Pay ADDITIONAL** energy and time to write back dirty page first
  - Later, may place in memory a page that was replaced
    - Paging energy and time for that replaced page is **paid AGAIN**
    - 👎👎

- **What if** "always" have an unoccupied page frame? 💡😀
  - No need to write back a dirty page!
  - Never have to re-load a page, because no page would need to be replaced!
  - Hmmm …  (lemon becomes lemonade)

Background – Information Theory

- **Information resolves uncertainty** (Claude Shannon, 1948)
  - The informational value of a message communicated using symbols from a given set is a function of the degree to which the message is **surprising**
  - Shannon says, information, H, should be measured as

    $H = - \sum p_i \log_2(p_i)$

    where p_i is probability of the i^{th} symbol in the set of available symbols

    H has units of *bits of information/symbol*

Information content examples

- Let Symbol ∈{heads, tails}. Map heads=1,tails=0. Symbol is a bit. Fair coin toss where $p_{head} = 0.5$, $p_{tail} = 0.5$
  - Info/toss, $H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$ bit/coin toss
  - **H = 1 bit of info/symbol (bit)**

- Let Symbol ∈ {0,1,…,9} embedded as unsigned integer in last 4 bits of a byte; assume probability of each digit is 0.1
  - $H = -\sum 0.1 \log_2(0.1) = 3.32$ bits of information/byte
  - **H = 0.415 bits of information/bit**
  - Why so little information/bit? Every symbol starts 0000,implies no surprise & no information; next 4 bits could convey 16 distinct symbols but encode only ten, so < 4 bits of surprise is possible

Compression of bit strings

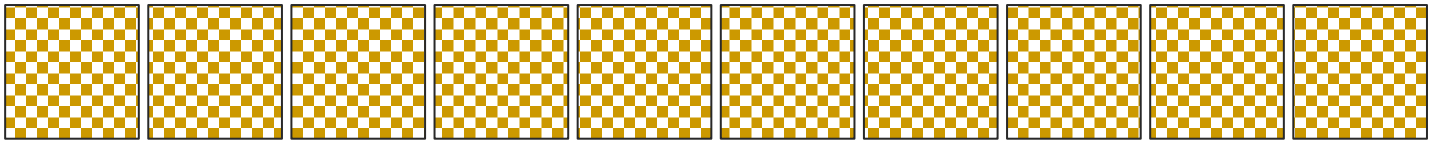
- Compression – appyling Shannon's insight
  - Find a symbol set where 1 bit/bit $\cong H_{compressed} > H_{original}$
  - **Shannon proved such a symbol set must exist**
  - After compression, message requires fewer bits
  - Invert compression function to access original bit string

- H for bit strings in computer memory
  - Typical page frame bit string has about 0.5 bit of info/ 1 bit of DRAM memory
  - ∴ content can be compressed into about ½ the space

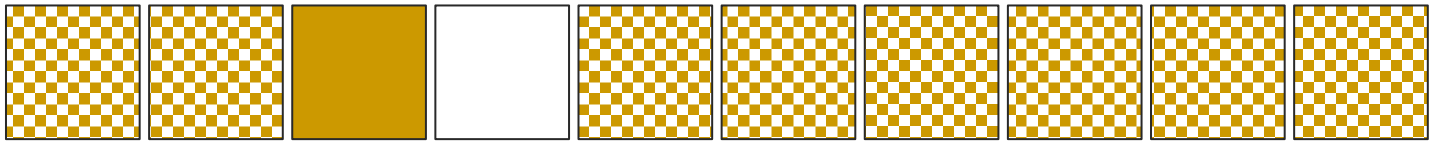
Apply Shannon's insight to virtual memory

- OS X Yosemite uses compression on page frames in DRAM to create *on demand* **unoccupied page frames**

  Uncompressed series of page frames,~0.5 bits/bit



  After compression of an adjacent pair of frames, an open frame appears next to a compressed frame with ~1 bit/bit of information; not usable until uncompressed



Idea 1: Create unoccupied frames *on demand*

- Use CPU time to "store" replaced pages in DRAM via compression
  - Compressing a frame takes far less time (~ 5,000 ns) than writing a page to disk (~10,000,000 ns) or to SSD
  - Compressing takes less energy than writing to disk
  - When need to access a compressed page, decompression designed to be fast, also
  - Replacement now occurs rarely
- Considerations for "compressed memory" software
  - Run in parallel on as many cores as available in CPU
  - Do not use virtual memory to track compressed page frames to avoid caching into TLB
    - Compression does not cause side-effect TLB cache misses for processes
- Can achieve approx. same result by spending 2x $ on DRAM

Idea 2: App Nap

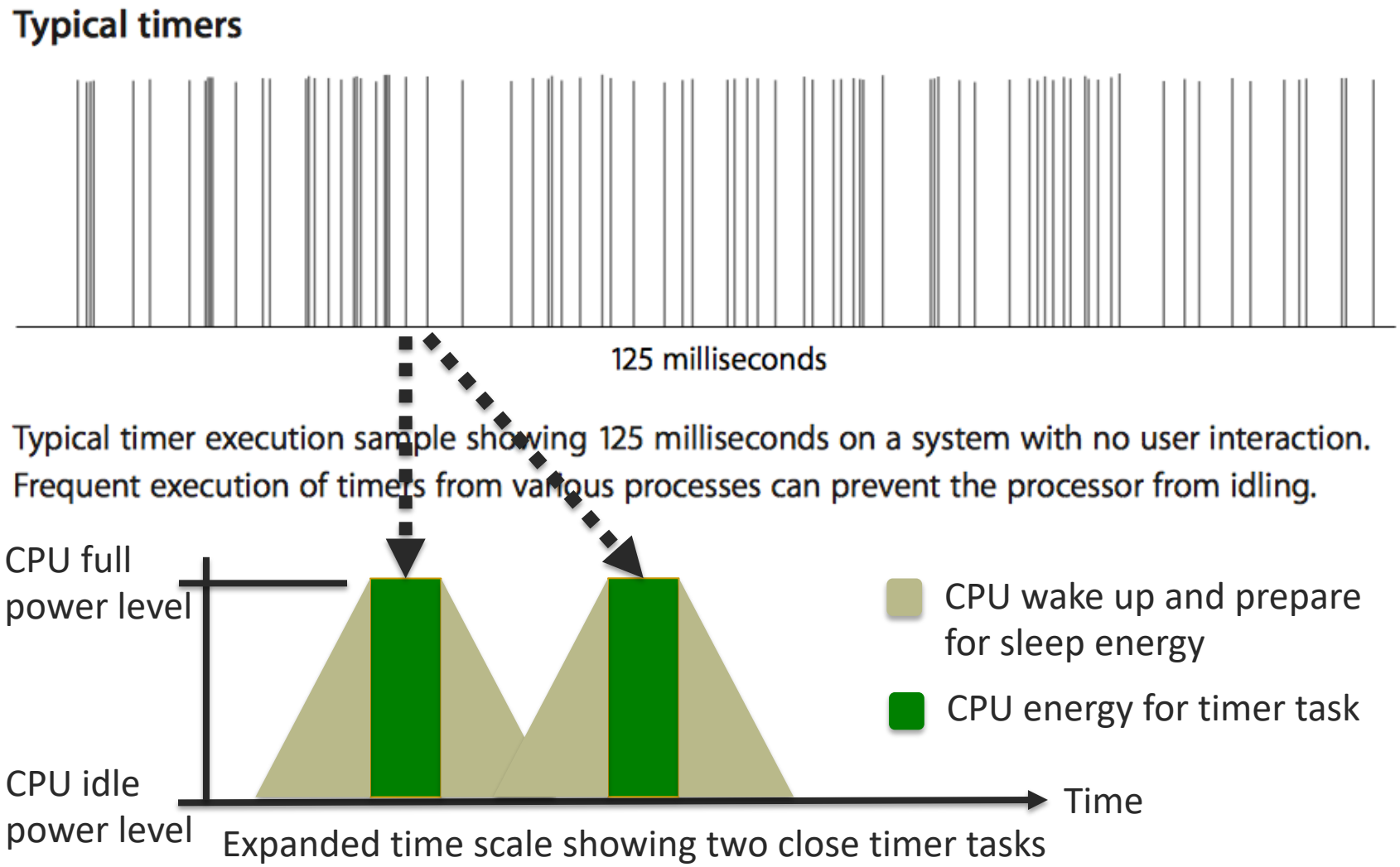
- Put App in state to reduce CPU and I/O use when
  1. its window is hidden, AND
  2. it is not playing audio
- Windows hidden by others are not updated, thus saving energy but need a moment to update when uncovered
- <humor>
  The sad, sad cost of App Nap 
  - No more *rear window defrost*

Best GUI feature name. Ever. </humor>

Idea 3: Intelligent timers

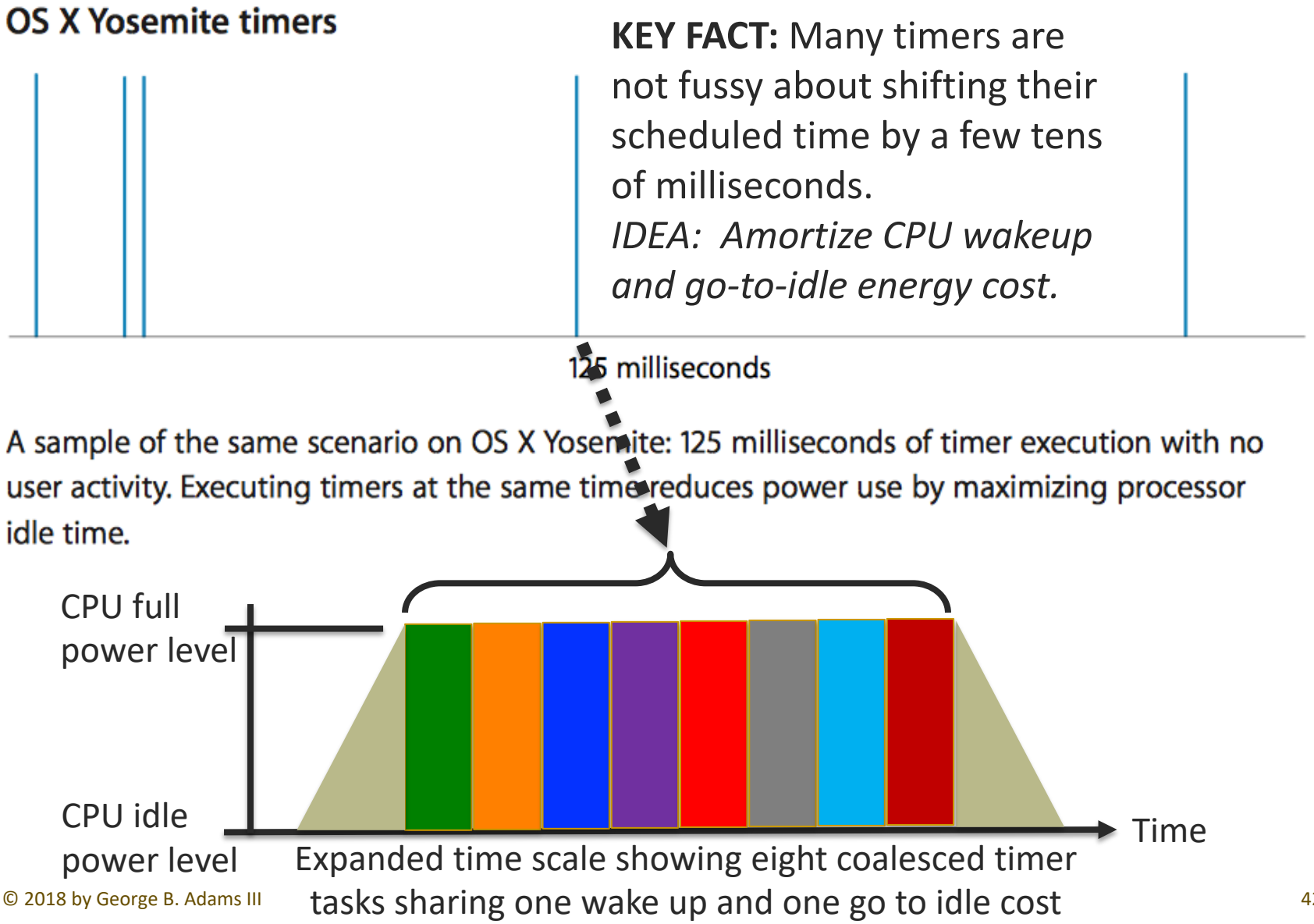
- Timers execute important tasks at intervals
- Timers cause a CPU to power up if idling
  - Powering up uses energy before execution begins
  - Entering idle state (powering down) uses energy other than for execution, also
- Bold idea: Find a way to do timer work while increasing the time spent idling
  - Work smarter, not longer

Timer timeline – original



Timer timeline – coalesced timers

From "OS X Yosemite Core Technologies Overview" Apple, Inc., Oct 2014, pp. 7 - 9.



Summary

- Examination of the entire computational platform from transistor up through the operating system can reveal opportunities for significant reductions in energy use and power demand
- Better designed software alone can enable significant energy savings without any change to hardware